APPENDIX / LIVE DEMO

A Walk Through the Code

The four stages of the loop in real code, and the commands that make the machines fix themselves on stage.

`tcpmon.c` `partition.go` `controller` `run-demo.sh`

Kept separate from the main deck. Show this only if there is time for the demo.

ORIENTATION

Where each part of the loop lives

WATCH	`internal/ebpf/bpf/tcpmon.c`	C (CO-RE)	Measures delay and speed of every connection
WATCH	`cmd/nodeagent`	Go	Loads the kernel programs, adds GPU state, reports it all
DECIDE	`internal/partition/partition.go`	Go	Works out the best cut, and decides when to apply it
ACT	`cmd/controller`	Go	Collects the readings, saves the plan, sends it out
HEAL	`internal/controller`	Go	Notices a dead machine in 3 seconds and forces a re-cut
SERVE	`worker/`	Python	Machines that hold layers but no state, plus the router
EVAL	`bench/run.py, bench/dpsim`	Python, Go	The tests: full cluster runs, and solver-only runs

The generated network code and the compiled kernel program are already in the repo, so a demo needs no extra build tools.

Measuring network delay inside the kernel

internal/ebpf/bpf/tcpmon.c

```
struct flow_val {
    __u64 bytes;         // cumulative payload bytes
    __u64 srtt_us;       // most recent smoothed RTT
    __u64 srtt_samples;  // cumulative tcp_probe hits
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key,   struct flow_key);
    __type(value, struct flow_val);
} flows SEC(".maps");

SEC("tp_btf/tcp_probe")
int BPF_PROG(tcp_probe_hook, struct sock *sk,
                             struct sk_buff *skb)
{
    struct tcp_sock *tp = (struct tcp_sock *)sk;
    __u32 srtt = BPF_CORE_READ(tp, srtt_us) >> 3;
    val->srtt_us = srtt;
    __sync_fetch_and_add(&val->srtt_samples, 1);
    return 0;
}

SEC("fentry/tcp_sendmsg")
int BPF_PROG(tcp_sendmsg_hook, struct sock *sk,
             struct msghdr *msg, size_t size)
```

Why we use the kernel's own number

Linux already tracks the round-trip time of every connection, because it needs it to manage traffic. Reading that number costs nothing, and it cannot be wrong the way a timer inside the program can.

Build once, run anywhere

The program adjusts itself to the running kernel when it is loaded. So one compiled file works on any modern Linux kernel, with no rebuilding.

We only watch what we need

We only look at connections between our own machines. The controller then matches each connection to the machine it came from.

Demo: show the live readings with kubectl logs

The solver that picks the cut

internal/partition/partition.go

```
// stageCost: receive + compute for one stage.
// An empty stage is skipped by the router entirely,
// so it costs nothing - not even its hop.
func stageCost(in Input, worker, layers int) float64 {
    if layers == 0 {
        return 0
    }
    cost := float64(layers) * in.PerLayerMs /
            math.Max(in.Workers[worker].Speed, 0.01)
    if worker > 0 {
        cost += in.LinkMs[worker-1]
    }
    return cost
}

// Optimal solves the linear partition in O(N^2 * K).
for w := 1; w <= k; w++ {
  for j := 0; j <= n; j++ {
    for split := 0; split <= j; split++ {
      cost := math.Max(f[split][w-1],
              stageCost(in, w-1, j-split))
      if cost < f[j][w] {
        f[j][w] = cost
        choice[j][w] = split
      }
    }
  }
}
```

f[j][w]

The best we can do for the first j layers on the first w machines. The final answer is f[N][K], and choice[][] tells us where the cuts go.

Heat is just a slower speed

Each machine has one Speed number. When it gets hot, that number drops. So the solver has no special code for heat anywhere - it just sees a slower machine.

A machine can get zero layers

That one line is what lets the solver skip a machine with a bad network. Nobody wrote a rule for skipping - it falls out of the maths.

Demo: go test ./internal/partition/... - it checks the solver against brute force on 200 random cases.

From a decision to a running system

internal/partition/partition.go - Decider

```
func (d *Decider) Decide(now time.Time, in Input,
                         force bool) (*Result, bool, error) {
    opt, err := Optimal(in)
    if d.current == nil || force ||
       workersChanged(d.current, in.Workers) {
        d.current, d.lastChange = opt, now
        return opt, true, nil          // healing path
    }
    currentCost := Evaluate(in, d.current.Splits())
    improved := opt.BottleneckMs <
                currentCost*(1-d.ImprovementFrac)
    if improved && now.Sub(d.lastChange) >= d.Cooldown {
        d.current, d.lastChange = opt, now
        return opt, true, nil          // hysteresis passed
    }
```

demo - make a machine hot, then cool it again

```
W2=$(docker inspect -f '{{range .NetworkSettings.Networks}}\
     {{.IPAddress}}{{end}}' kubeedgeinfer-worker2)
curl -X POST http://$W2:9101/gpu/override -d '{"temp_c": 92}'
curl -X POST http://$W2:9101/gpu/override -d '{"clear": true}'
```

Three ways out, one function

If a machine appears or dies, re-cut at once. If a new cut is clearly better and enough time has passed, apply it. Otherwise keep the current cut and just update its cost.

We compare fairly

Before comparing, we re-score the cut we are already using with today's readings. Otherwise we would be comparing against an old number.

Changing the cut with no restart

The controller saves the new layer ranges in Kubernetes and sends them to the machines with a new version number. Machines refuse messages from an old version. The router then fetches the new plan and replays the request from the start. Machines keep no state, so replaying is always safe.

Layers move off the hot machine in about 35 seconds, and come back once it cools down.

DEMO

Runbook

1 One command starts everything

```
./run-demo.sh
```

Builds everything, starts the cluster, and opens a live dashboard on localhost:8000 showing the machines, how busy each one is, the network delays, and a log of every re-cut.

2 Watch the plan change

```
kubectl -n kubeedgeinfer get ipl demo -w
```

The layer ranges and the version number update as the controller re-plans.

3 Send it some work

```
kubectl -n kubeedgeinfer port-forward svc/router 8080:8080
curl -X POST localhost:8080/generate \
  -d '{"prompt_len":16,"max_new_tokens":8}'
```

The router sends each request through the machines in order.

4 Break something on purpose

```
curl -X POST http://$W2:9101/gpu/override \
  -d '{"temp_c": 92}'
```

Or press the buttons on the dashboard. Layers move off the hot machine, then come back once it cools.

5 Reproduce the numbers

```
go run ./bench/dpsim
python3 bench/run.py --all && python3 bench/plot.py
```

dpsim reproduces the numbers on slide 11 with no cluster at all. bench/run.py needs the running cluster and measures words per second and idle time.

You need: a modern Linux kernel, Docker, kind, kubectl and Python 3.11 or newer.